СОРТИРОВКА
СЛИЯНИЕМ

Created by Ахметов Игорь
2026

В чем смысл?

Идея слияния (англ. merge) используется очень часто.
Если есть две отсортированные последовательности, то результатом слияния должна быть отсортированная последовательность.

Ключевые особенности:

- Гарантированное время $O(n\log(n))$
- Стабильная сортировка (сохраняет порядок равных элементов).
- Требует дополнительной памяти $O(n)$.

А в чем идея?

1. Разделение (Divide): Рекурсивно делим массив пополам, пока не получим подмассивы размером 1.
2. Властвование (Conquer): Сортируем полученные части (база рекурсии: массив из 1 элемента уже отсортирован).
3. Комбинирование (Combine/merge): Сливаем два отсортированных подмассива в один большой отсортированный массив.

РАЗДЕЛЯЙ И ВЛАСТВУЙ!!!

Не псевдо-алгоритм на псевдо-коде

Алгоритм 15: Слияние двух отсортированных подмассивов

Вход: A — массив, l — левая граница массива, r — правая граница массива, m — место разбиения на подмассивы. Подмассивы $[l, m]$ и $[m+1, r]$ являются отсортированными.

Выход: Отсортированный подмассив $[l, r]$

начало алгоритма

- **если** $l \geqslant r$ *или* $m < l$ *или* $m > r$ **то**
  - · прекращаем работу алгоритма;
- **если** *в массиве 2 элемента и они не отсортированы* **то**
  - · меняем их местами;
  - · прекращаем работу алгоритма;
- · Создаем дополнительный массив buf размерностью $r - l + 1$;
- · $cur = 0$;
- · $i = l, j = m + 1$;
- **цикл пока** $r - l + 1 \neq cur$ **выполнять**
  - **если** $i > m$ **то**
    - · дописываем в buf оставшиеся элементы от j до r ;
  - **иначе если** $j > r$ **то**
    - · дописываем в buf оставшиеся элементы от i до m ;
  - **иначе если** $a[i] > a[j]$ **то**
    - · записываем в buf $a[j]$;
    - · увеличиваем j;
  - **иначе**
    - · записываем в buf $a[i]$;
    - · увеличиваем i;
- · записываем массив buf в начальный массив A, начиная с индекса l;

конец алгоритма

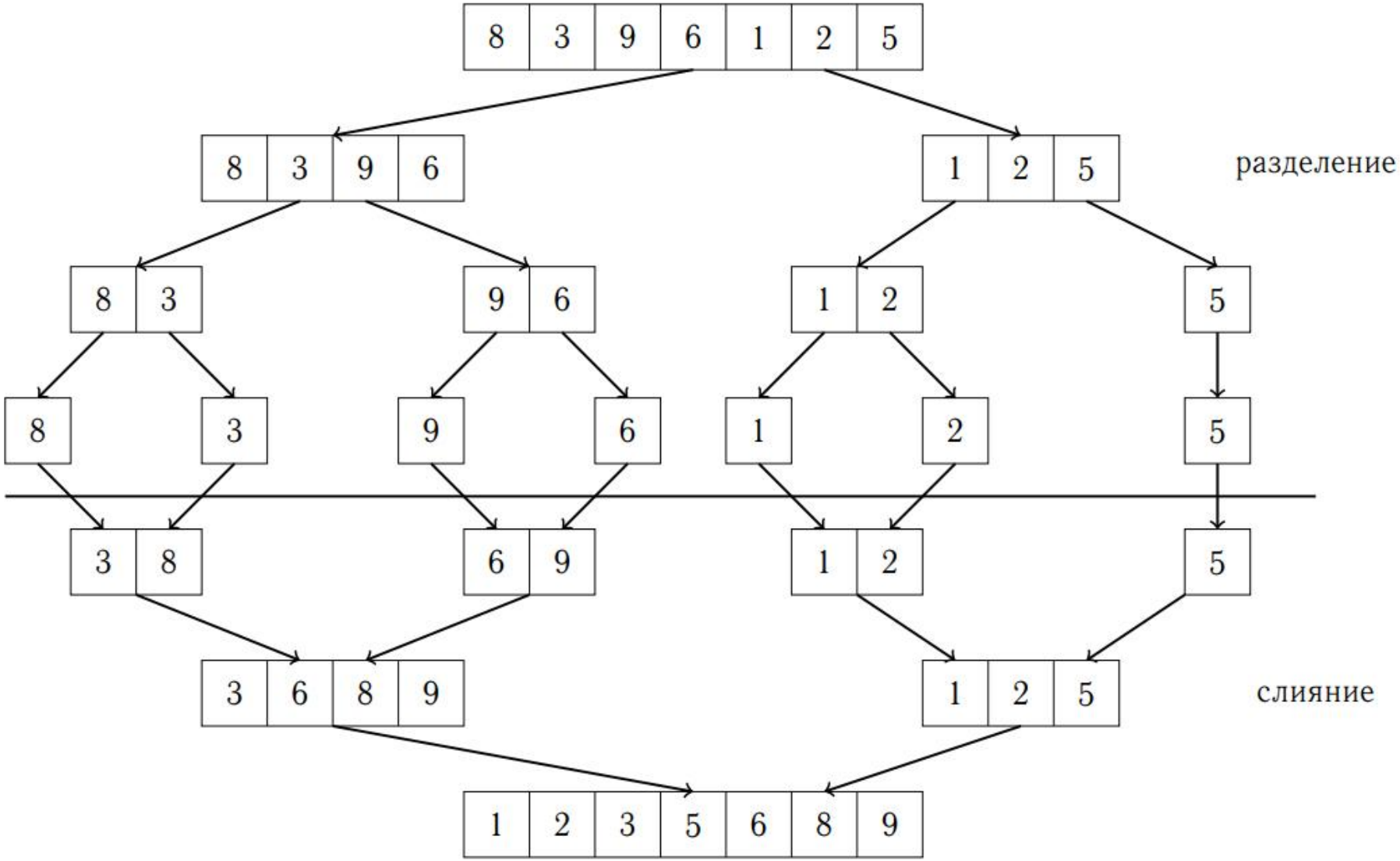
Еще немного псевдо-кода

Алгоритм 16: Сортировка слиянием

Вход: A — массив, l — левая граница массива, r — правая граница массива
Выход: Отсортированный массив
начало алгоритма
  если $l \geqslant r$ **то**
    · прекращаем работу алгоритма;
  · $m = \frac{l+r}{2}$;
  · вызываем данную функцию от (A, l, m);
  · вызываем данную функцию от $(A, m+1, r)$;
  · вызываем функцию слияния от (A, l, r, m);
конец алгоритма

Пример работы алгоритма

На входе имеем неотсортированный массив



Реализация работы сортировки

```
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    // выделяем память для временных массивов
    int* L = new int[n1];
    int* R = new int[n2];
    // копируем данные
    for (int i = 0; i < n1; ++i) {
        L[i] = arr[left + i];
    }
    for (int j = 0; j < n2; ++j) {
        R[j] = arr[mid + 1 + j];
    }
    // слияние временных массивов обратно в arr
    int i = 0; // для L
    int j = 0; // для R
    int k = left; // для результирующей части arr
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        }
        else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
```

```
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    delete[] L;
    delete[] R;
}

// Сама рекурсивная функция сортировки
void mergeSort(int arr[], int left, int right) {
    if (left >= right) {
        return;
    }
    int mid = left + (right - left) / 2;

    mergeSort(arr, left, mid); // 1 половина
    mergeSort(arr, mid + 1, right); // 2 половина

    // слияние отсортированных половин
    merge(arr, left, mid, right);
}
```

Используйте merge sort - не живите в беспорядке!

Спасибо за внимание!!!